EdgeMATX

Design and Integration of a TinyML Systolic Accelerator with PicoRV32

Submitted by

Nishchay Pallav 221EC233

Mohammad Omar Sulemani 221EC230

Md Atib Kaif 221EC129

Under the Guidance of Dr Rathamala Rao

Dept. of Electronics & Communication Engineering
National Institute of Technology Karnataka, Surathkal 575025

673
Cycles

54x
Peak Speedup

19/19
Tests Pass

4x4
Systolic Array

Introduction

A hardware coprocessor for matrix multiplication on embedded RISC-V

What is EdgeMATX?

A custom hardware coprocessor tightly coupled to the open-source PicoRV32 RISC-V processor. A single custom instruction replaces 64 sequential software operations with 16 fully parallel hardware MACs.

Why Does It Matter?

> TinyML inference on edge devices is dominated by dense matrix operations

> General-purpose RISC-V cores execute these sequentially -- very slowly

> Purpose-built systolic hardware delivers 38-54x speedup at minimal area

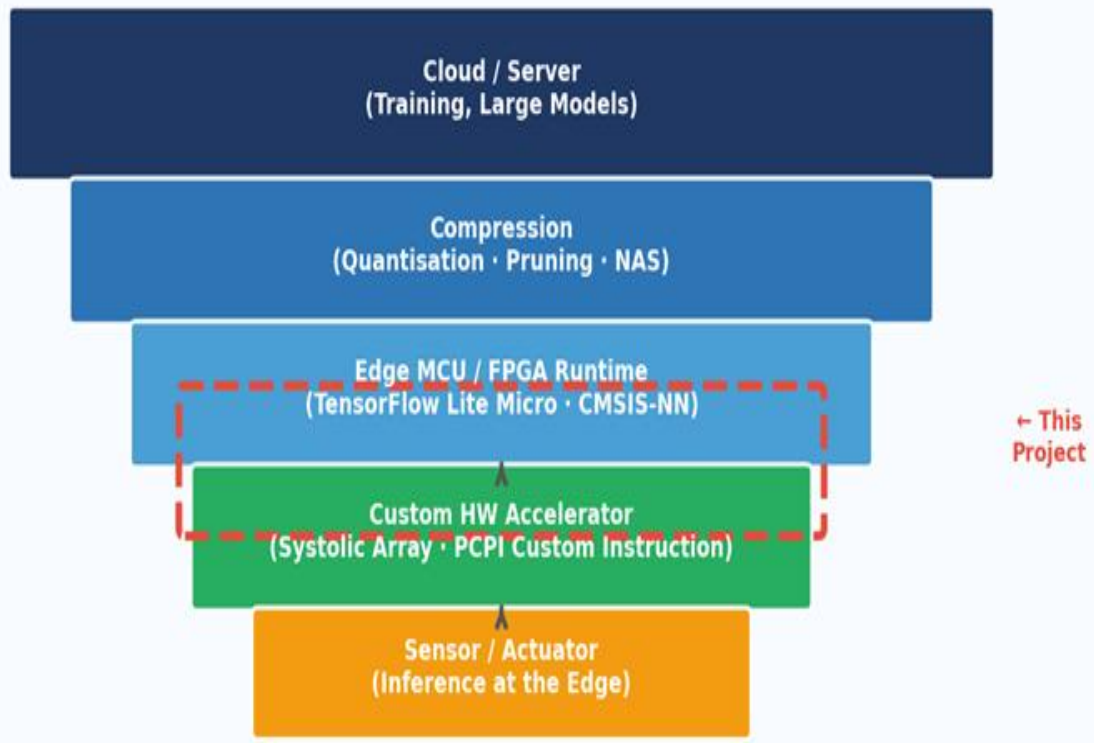
Our Approach

One opcode. CPU stalls. 4x4 systolic array fires. Results appear in memory. 673 deterministic cycles -- every time.

Scope & Status

RTL simulation complete. 19/19 tests passing. Two live web visualizers. FPGA deployment upcoming.

Figure 4 - TinyML System Stack: this project targets the hardware accelerator layer



The Problem

Matrix multiplication is ubiquitous -- and painfully slow on embedded CPUs

Where It's Needed

TinyML Inference
Neural network layers are dense GEMM ops

Digital Signal Proc.
Convolution = repeated matrix multiply

Scientific Computing
Linear solvers, transforms, all matrix ops

Image Processing
Filters, transforms, feature extraction

Sequential CPU Execution

```
for i in range(4):
  for j in range(4):
    for k in range(4):
      C[i][j] += A[i][k]
            * B[k][j]
```

64 multiply-accumulate ops
executed one at a time

RV32I (no MUL): 26,130 cycles
RV32IM (with MUL): 7,975 cycles

The Gap

26 K
cycles wasted per
4x4 multiply

100%
sequential
execution on CPU

0x
hardware
parallelism

The Solution: EdgeMATX

A custom PCPI coprocessor tightly coupled to PicoRV32

One custom instruction replaces 64 sequential CPU operations

<= Tightly Coupled

PCPI interface: zero memory-mapped overhead. CPU stalls and resumes -- no interrupts, no DMA.

[+] Systolic Datapath

4x4 array of 16 processing elements. All 16 MACs execute in parallel every clock cycle.

Q5.10 Fixed-Point

16-bit signed arithmetic. No floating-point hardware needed -- ideal for constrained silicon.

673	Deterministic cycles	**38.83x**	Speedup (sparse data)	**53.86x**	Speedup (dense data)	**11.85x**	Speedup vs MUL rv32im

Project Progress

Simulation-complete milestone -- all RTL objectives achieved

DONE

Phase 1: RTL Design

[DONE] 4x4 systolic array in Verilog (Q5.10 fixed-point MACs)
[DONE] PCPI coprocessor wrapper for PicoRV32
[DONE] Memory-mapped I/O: A@0x100 B@0x140 C@0x200

DONE

Phase 2: Simulation & Verification

[DONE] 19/19 standalone accelerator tests -- 100% PASS
[DONE] 8/8 CPU integration regression tests -- 100% PASS
[DONE] Cycle benchmarking: 673 deterministic cycles confirmed

DONE

Phase 3: Tooling & Documentation

[DONE] Two interactive web visualizers deployed on Vercel
[DONE] Full LaTeX project report submitted
[DONE] 38-54x speedup measured and documented

NEXT

Phase 4: FPGA Deployment

[ ] Synthesize on Pynq-Z2 (Xilinx Zynq-7020)
[ ] On-board cycle validation and timing closure
[ ] AXI-BRAM memory interface integration

System Architecture

Firmware -> PicoRV32 -> PCPI -> 4x4 Systolic Accelerator -> Memory

Architecture Layers

Firmware (RISC-V C)

Loads matrices at mapped addresses, issues custom opcode 0x5420818b

PicoRV32 CPU

RV32I core, ENABLE_PCPI=1, decodes custom opcode, stalls until pcpi_ready

PCPI Wrapper

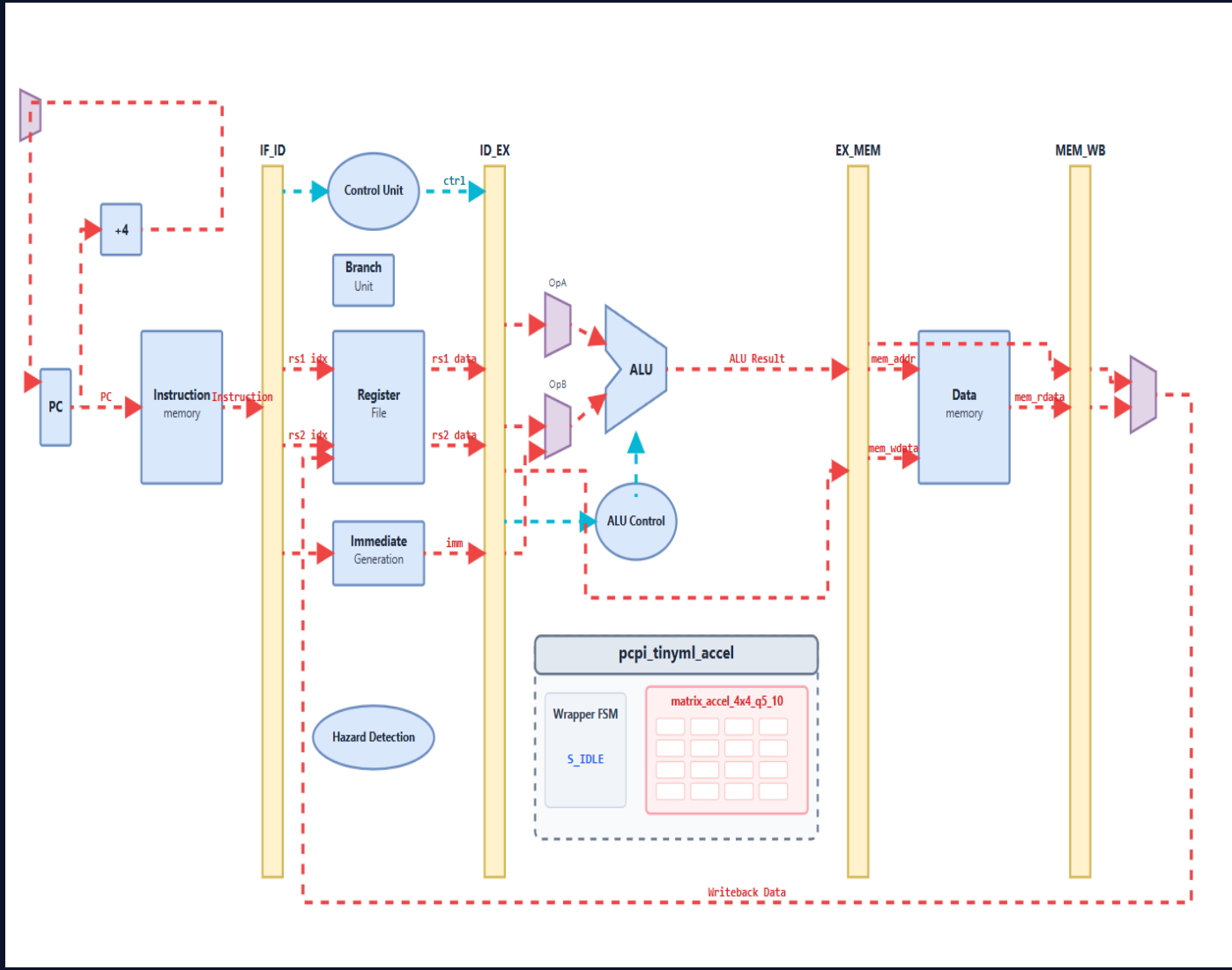
Detects instruction, drives systolic engine, asserts pcpi_ready on finish

4x4 Systolic Array

16 Q5.10 MAC units, output-stationary dataflow, 673 deterministic cycles

Flat Memory Map

A: 0x100-0x13C B: 0x140-0x17C C (result): 0x200-0x23C



PCPI: The Custom Instruction Bridge

Pico Co-Processor Interface -- zero-cost hardware dispatch

Interface Signals

pcpi_valid
CPU -> CoPro | Custom instruction detected

pcpi_insn
CPU -> CoPro | Full 32-bit instruction word

pcpi_rs1/2
CPU -> CoPro | Source registers (matrix A/B addr)

pcpi_ready
CoPro -> CPU | Computation done -- CPU resumes

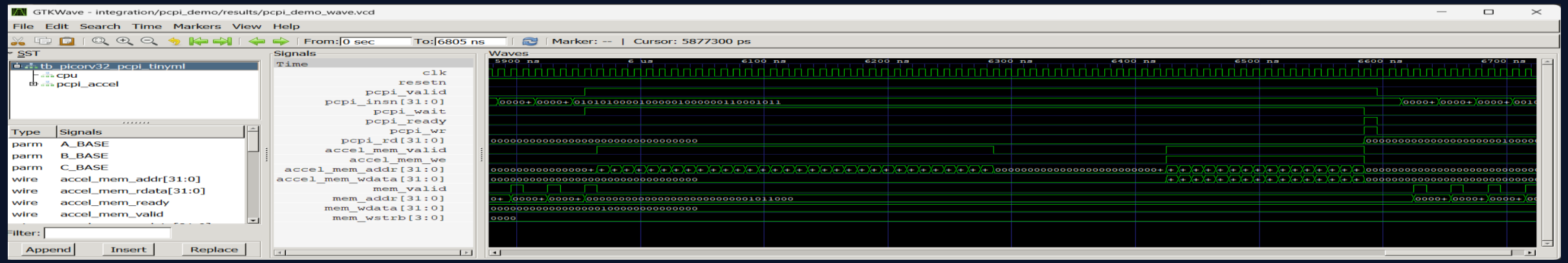
pcpi_wr/rd
CoPro -> CPU | Write-back flag + result (unused)

pcpi_wait

Execution Handshake

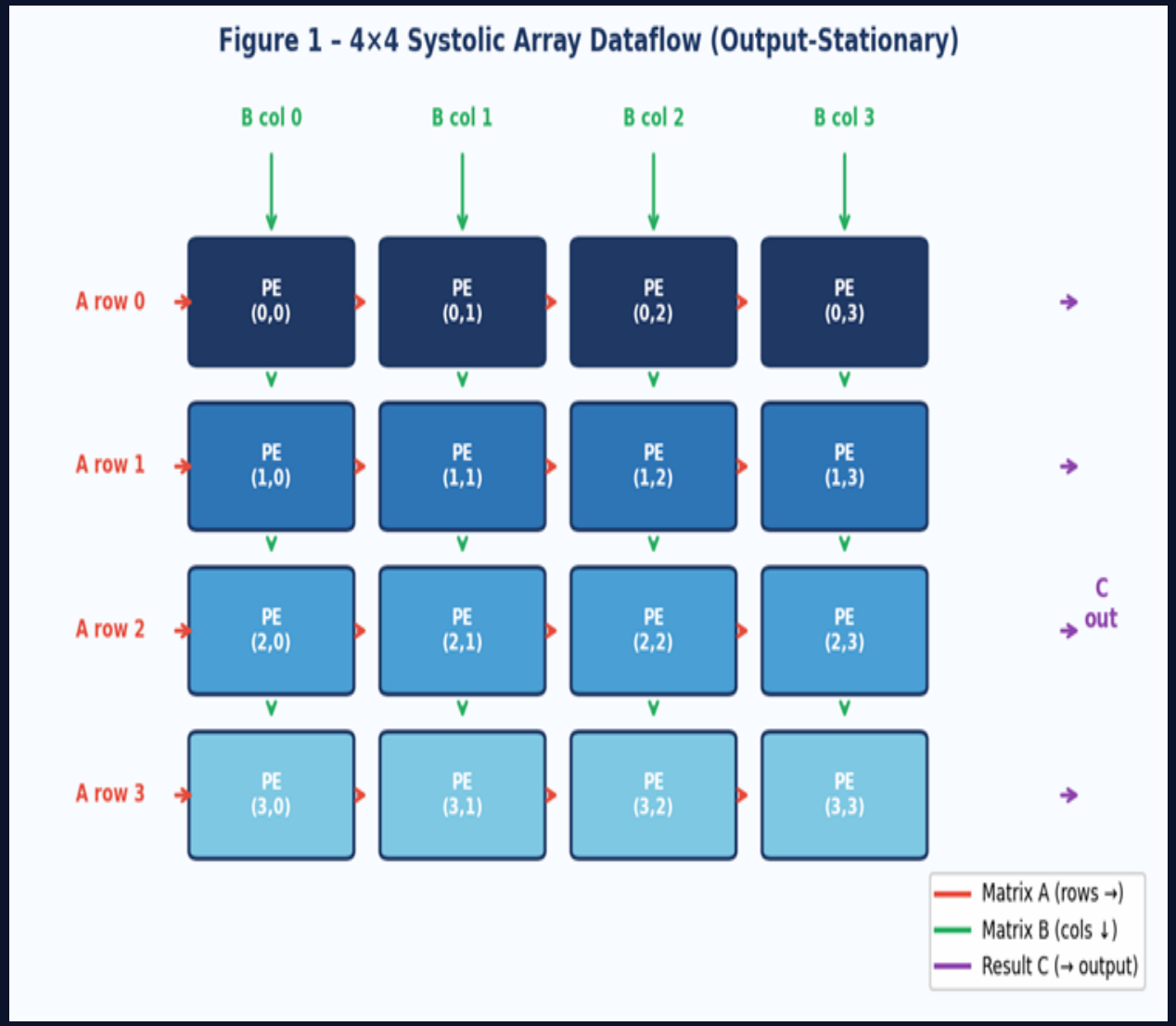
1. CPU fetches 0x5420818b custom opcode
2. pcpi_valid=1, rs1/rs2 broadcast to coprocessor
3. CPU stalls -- awaiting pcpi_ready=1
4. Coprocessor reads matrix A & B from memory
5. 4x4 systolic array computes 16 dot products
6. Results written to matrix C region (0x200-0x23C)
7. pcpi_ready=1 -- CPU resumes next instruction

Custom opcode: 0x5420818b (custom-0 space, funct7=0101010, rs2=00000, funct3=000, opcode=0001011)



The 4x4 Systolic Array

16 parallel Processing Elements -- output-stationary dataflow



How It Works

Output-stationary
Each PE accumulates its partial sum for C[i,j] in place

Row broadcast
Row i of matrix A flows right across row i of PEs

Column broadcast
Column j of matrix B flows down column j of PEs

Parallel MACs
All 16 PEs fire simultaneously every clock cycle

Deterministic
673 cycles regardless of operand values

16-bit Q5.10
Multiply -> shift-right 10 -> accumulate -> truncate

Q5.10 Fixed-Point Arithmetic

16-bit two's complement -- integer simplicity, fractional precision

S	I4	I3	I2	I1	I0	F9	F8	F7	F6	F5	F4	F3	F2	F1	F0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0

Sign (1 bit)

Integer part (5 bits)

Fractional part (10 bits)

Range

-32.0 to +31.999

Resolution

1 / 1024 ~ 0.001

Format

16-bit signed integer

Scaling

Multiply -> shift right 10 bits

Processing Element MAC Operation (per clock cycle):

(1) Multiply

a_in x b_in -> 32-bit signed product

(2) Scale

product >>> 10 (arithmetic right-shift corrects Q5.10)

(3) Accumulate

acc += scaled_product (32-bit running sum)

(4) Pass through

a_out <- a_in ; b_out <- b_in (feed neighbours)

Verification Strategy

Four-layer simulation campaign -- unit tests to full system benchmarking

4 **Layer 4 | Performance Benchmarking**

Cycle-count measurement against two software baselines (no-MUL and MUL-equipped).

673 cycles

3 **Layer 3 | Protocol Handoff Validation**

Isolated test of PCPI signal handshake -- pcpi_valid, pcpi_ready, pcpi_insn correctness.

PASS

2 **Layer 2 | CPU Integration Regression**

Full system simulation: PicoRV32 + PCPI wrapper + accelerator. 8 input profiles.

8 / 8 PASS

1 **Layer 1 | Standalone Unit Tests**

Accelerator module in isolation. Zero, identity, dense, boundary, negative-element cases.

19 / 19 PASS

Verification Results

100% pass rate across all test flows -- simulation-complete milestone

100%

PASS RATE

19 / 19 standalone unit tests

8 / 8 integration cases

5 / 5 professor demo cases

PASS handoff protocol test

PASS one-command gate check

Test Category Breakdown

Performance: Cycle Counts

Deterministic execution -- 673 cycles regardless of input values

673

EdgeMATX Accelerator

rv32i + PCPI

7,975

Software + MUL

rv32im baseline

26,130

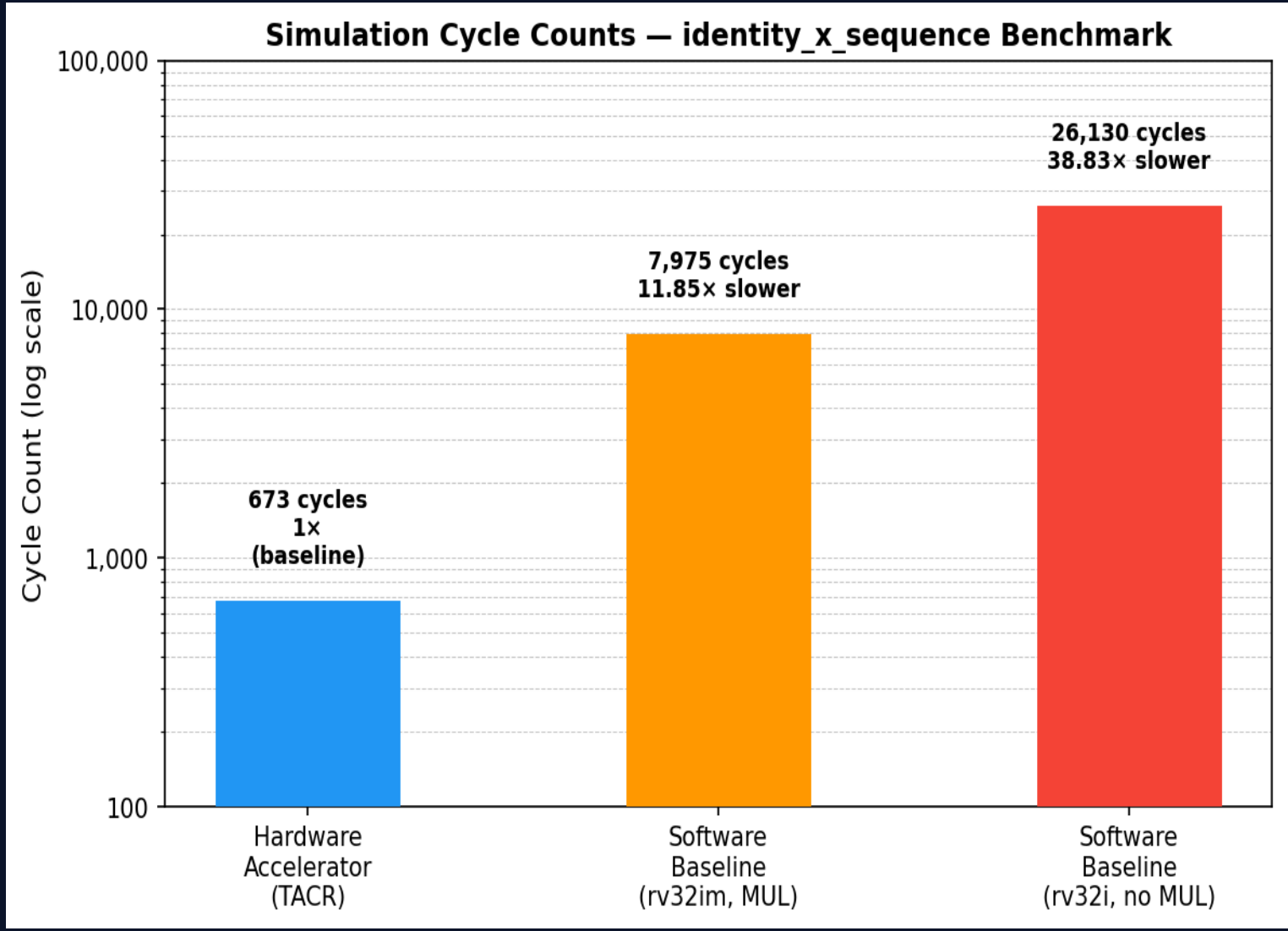
Software (no MUL)

rv32i baseline

36,246

Software (no MUL, dense)

dense input profile



Performance: Speedup

38-54x over rv32i baseline -- 11.85x over MUL-equipped core

38.83x

Sparse vs rv32i

Identity-sequence benchmark
26,130 -> 673 cycles

53.86x

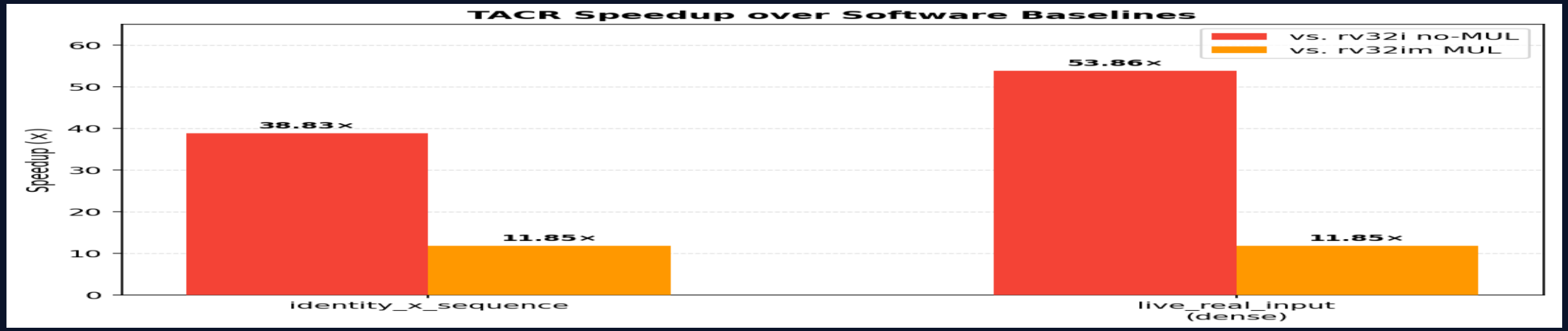
Dense vs rv32i

Live dense profile
36,246 -> 673 cycles

11.85x

Any vs rv32im

Even against MUL-equipped core
7,975 -> 673 cycles



Interactive Visualizers

Explore the design live in your browser -- click links below

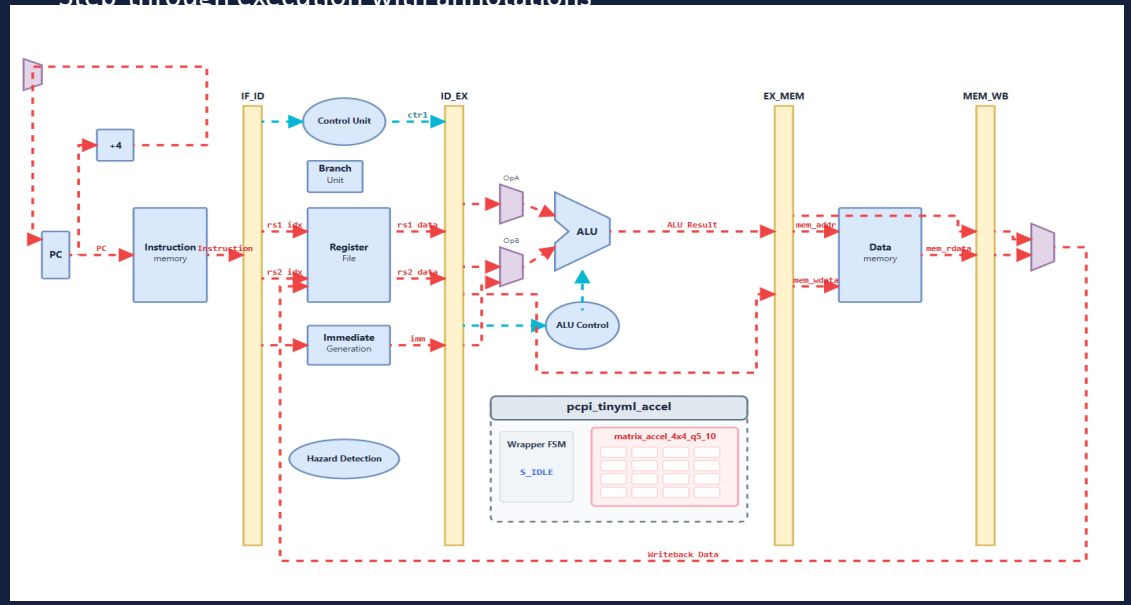
Architecture Overview

riscv-visualizer-2.vercel.app

High-level system architecture and dataflow

Firmware layer | PicoRV32 | PCPI | Systolic Array

Step-through execution with annotations



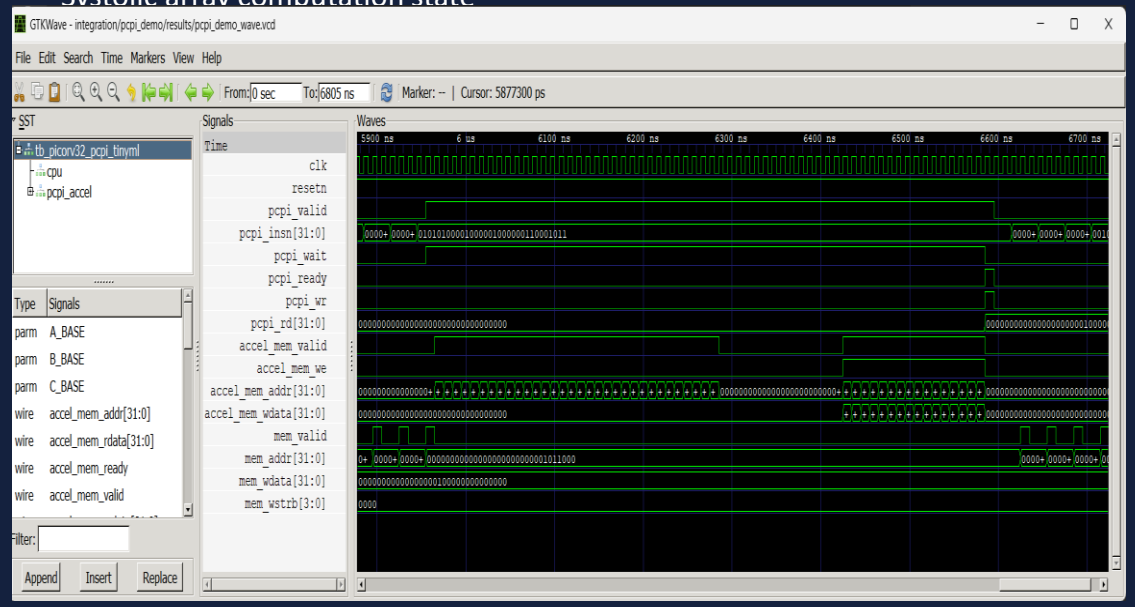
Processor Signal Simulation

tinyml-pcpi-visualizer.vercel.app

Instruction decode | PCPI signal timeline

Accelerator invocation waveform

Systolic array computation state



What's Next: Pynq-Z2 FPGA Deployment

Simulation-proven design -> physical hardware closure

DONE

Simulation

673-cycle verified
behaviour on
Icarus Verilog

NEXT

FPGA Synthesis

Map to Pynq-Z2
Xilinx Zynq-7020
Post-synthesis timing

FUTURE

Scale & Deploy

On-board cycle counts
AXI-BRAM memory
8x8 array extension

Open Items Before Hardware Closure

- Cycle-scaling estimator anchor needs updating (869 -> 673)
- On-board rdcycle CSR measurements to cross-validate simulation results
- AXI-BRAM memory interface replacing software-driven memory reads

Conclusion

Simulation-complete milestone -- all objectives achieved

Hardware Accelerator

4x4 systolic array with 16 Q5.10 fixed-point MACs, implemented in Verilog

CPU Integration

Tightly coupled to PicoRV32 via PCPI -- single opcode dispatch, zero overhead

Deterministic Execution

673-cycle latency, invariant to operand values -- ideal for real-time systems

Verified Performance

38.83x - 53.86x speedup over rv32i; 11.85x over MUL-equipped rv32im

100% Test Coverage

19/19 standalone | 8/8 integration | 5/5 professor demo | handoff PASS

Interactive Tooling

Two live web visualizers for architecture walkthrough and signal-level simulation

Thank You

Questions & Discussion

Team

Nishchay Pallav 221EC233

Mohammad Omar Sulemani 221EC230

Md Atib Kaif 221EC129

Guide: Dr Rathamala Rao

Live Visualizers

riscv-visualizer-2.vercel.app

tinyml-pcpi-visualizer.vercel.app

Dept. of Electronics & Communication Engineering
National Institute of Technology Karnataka, Surathkal 575025 | 2026